```
# Install (run once)
# pip install torch transformers datasets peft accelerate

import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset
from peft import LoraConfig, get_peft_model
from transformers import TrainingArguments, Trainer
```

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%    483/483 [00:00<00:00, 7.31kB/s]

tokenizer_config.json: 100%    48.0/48.0 [00:00<00:00, 996B/s]

vocab.txt:    232k/? [00:00<00:00, 2.52MB/s]

tokenizer.json:    466k/? [00:00<00:00, 2.74MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
model.safetensors: 100%    268M/268M [00:03<00:00, 155MB/s]

Loading weights: 100%    100/100 [00:00<00:00, 198.23it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
classifier.weight       | MISSING    |
pre_classifier.weight   | MISSING    |
pre_classifier.bias     | MISSING    |
classifier.bias         | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

```
dataset = load_dataset("imdb")

def tokenize_function(example):
    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length",
        max_length=256
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)

train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(200))

train_dataset = train_dataset.remove_columns(["text"])
test_dataset = test_dataset.remove_columns(["text"])

train_dataset.set_format("torch")
test_dataset.set_format("torch")
```

```
README.md:    7.81k/? [00:00<00:00, 169kB/s]
plain_text/train-00000-of-00001.parquet: 100%    21.0M/21.0M [00:00<00:00, 108MB/s]
plain_text/test-00000-of-00001.parquet: 100%    20.5M/20.5M [00:00<00:00, 103MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%    42.0M/42.0M [00:00<00:00, 209MB/s]
Generating train split: 100%    25000/25000 [00:00<00:00, 45105.19 examples/s]
Generating test split: 100%    25000/25000 [00:00<00:00, 66409.77 examples/s]
Generating unsupervised split: 100%    50000/50000 [00:00<00:00, 83364.07 examples/s]
Map: 100%    25000/25000 [00:41<00:00, 902.34 examples/s]
Map: 100%    25000/25000 [00:33<00:00, 736.64 examples/s]
Map: 100%    50000/50000 [01:16<00:00, 789.08 examples/s]
```

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_lin", "v_lin"],
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)
```

```
peft_model = get_peft_model(model, lora_config)

peft_model.print_trainable_parameters()
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no",
    report_to="none"
)

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
)

trainer.train()
```

[39/63 05:23 < 03:29, 0.11 it/s, Epoch 0.60/1]

Step	Training Loss
10	0.331390
20	0.398037
30	0.473798

[63/63 08:32, Epoch 1/1]

Step	Training Loss
10	0.331390
20	0.398037
30	0.473798
40	0.291151
50	0.366001
60	0.274011

```
TrainOutput(global_step=63, training_loss=0.36081884966956246, metrics={'train_runtime': 525.6225,
'train_samples_per_second': 0.951, 'train_steps_per_second': 0.12, 'total_flos': 33684851712000.0, 'train_loss':
```

```
results = trainer.evaluate(eval_dataset=test_dataset)
print(results)
```

[2/25 00:02 < 01:04, 0.36 it/s]

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no",
    report_to="none"
)

trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
)

trainer.train()
```

[63/63 07:56, Epoch 1/1]

Step	Training Loss
10	0.238476
20	0.428488
30	0.488508
40	0.238075
50	0.351535
60	0.272573

```
TrainOutput(global_step=63, training_loss=0.3435492856161935, metrics={'train_runtime': 499.7857,
'train_samples_per_second': 1.0, 'train_steps_per_second': 0.126, 'total_flos': 33684851712000.0, 'train_loss':
```

```
results = trainer.evaluate(eval_dataset=test_dataset)
print(results)
```

[25/25 01:15]

```
{'eval_loss': 0.45012006163597107, 'eval_runtime': 82.9168, 'eval_samples_per_second': 2.412, 'eval_steps_per_second': 0.302
```